

Will They Use It or Not? Investigating Software Developers' Intention to Follow Privacy Engineering Methodologies

AWANTHIKA SENARATH, University of New South Wales Australia
MARTHIE GROBLER, data61, Commonwealth Scientific and Industrial Research Organisation, Australia
NALIN ASANKA GAMAGEDARA ARACHCHILAGE, La Trobe University Australia and University of New South Wales

With the increasing concerns over privacy in software systems, there is a growing enthusiasm to develop methods to support the development of privacy aware software systems. Inadequate privacy in software system designs could result in users losing their sensitive data, such as health information and financial information, which may cause financial and reputation loss. Privacy Engineering Methodologies (PEMs) are introduced into the software development processes with the goal of guiding software developers to embed privacy into the systems they design. However, for PEMs to be successful it is imperative that software developers have a positive intention to use PEMs. Otherwise, developers may attempt to bypass the privacy methodologies or use them partially and hence develop software systems that may not protect user privacy appropriately. To investigate the factors that affect software developers' behavioural intention to follow PEMs, in this article, we conducted a study with 149 software developers. Findings of the study show that the usefulness of the PEM to the developers' existing work to be the strongest determinant that affects software developers' intention to follow PEMs. Moreover, the compatibility of the PEM with their way of work and how the PEM demonstrates its results when used were also found to be significant. These findings provide important insights in understanding the behaviour of software developers and how they perceive PEMs. The findings could be used to assist organisations and researchers to deploy PEMs and design PEMs that are positively accepted by software developers.

CCS Concepts: • **Computer systems organisation** → **Embedded systems**; *Redundancy*; Robotics; • **Networks** → Network reliability;

Additional Key Words and Phrases: Privacy engineering, software development, technology acceptance

ACM Reference format:

Awanthika Senarath, Marthie Grobler, and Nalin Asanka Gamagedara Arachchilage. 2019. Will They Use It or Not? Investigating Software Developers' Intention to Follow Privacy Engineering Methodologies. *ACM Trans. Priv. Secur.* 22, 4, Article 23 (October 2019), 30 pages.
<https://doi.org/10.1145/3364224>

Authors acknowledge the support given by the Information Management and Technology Team of XXXX Australia in setting up and providing the servers for the survey.

Authors' addresses: A. Senarath; email: a.senarath@student.unsw.edu.au; M. Grobler; email: marthie.grobler@data61.csiro.au; N. A. G. Arachchilage; email: nalin.n.arachchilage@latrobe.edu.au.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2019 Association for Computing Machinery.

2471-2566/2019/10-ART23 \$15.00

<https://doi.org/10.1145/3364224>

1 INTRODUCTION

With the pervasiveness of technology and software systems that we use on a daily basis (such as social networking apps, health monitoring apps, etc.), our data is not isolated anymore [51]. Rather, data has become an integral part of software systems, stored electronically in places not known to the data owner [74]. Therefore, if the systems do not have appropriate levels of protection for data, users could easily lose their data to unknown parties [51, 82]. This leads to systems being privacy invasive [79]. Therefore, there are increasing concerns over data protection in software systems with scrutinized data protection regulations [2, 43] and user concerns [15].

However, developers are not privacy experts. They do not have the expertise and the knowledge required to embed the legal and social requirements for data protection into software systems [16, 33, 76]. This has resulted in a need within software development organisations to integrate Privacy Engineering Methodologies (PEMs) into software development processes. These PEMs are expected to guide and enable software developers to implement data protection in software systems [27].

Introduction of a PEM into a software development process is an expensive endeavour in terms of money, time, and effort [30]. Therefore, it is imperative that developers practice these PEMs effectively within the software development process to embed privacy into the software systems.

Organisations that attempt to deploy any methodology in software development processes often face significant resistance from individual software developers [32, 42, 54, 61]. Even though the use of a system development methodology or a process improvement methodology is an organisational decision, its use is usually determined by the developers who use it. For example, previous research has found that only 6% of organisations that have methodologies established in software development processes claim that their methodologies are successfully used as expected by developers [23]. This resistance could be far worse for PEMs, because developers usually find it challenging to use privacy guidelines at work because of the vague and ambiguous privacy instructions and lack of compatibility with their usual development practices [8, 27, 63, 67]. Developers considering privacy to be a secondary or an irrelevant task in software development [44, 65] could also make the deployment of PEMs in a deployment process more difficult. When developers resist following PEMs at work, they may develop software systems that have privacy vulnerabilities, which could risk users' privacy and the organisation's reputation and money.

Therefore, investigating what makes developers follow, or reject, a PEM is important. This could help organisations to better facilitate the process of deploying their PEMs to avoid developers bypassing or rejecting the PEM. Privacy Engineers, researchers, and managers could benefit from knowing what aspects they need to consider when designing a PEM for software developers to make developers have a positive intention to use the PEM. However, so far there has been no systematic study that investigates the factors that affect software developers' acceptance or rejection of PEMs.

In this study, we empirically investigate the intention of developers to use PEMs. Based on existing knowledge [31, 73] and the Technology Acceptance Model (TAM) [19, 20], we use the constructs of perceived usefulness, perceived complexity, perceived compatibility, results demonstrability, and the perceived social pressure to measure software developers' behavioural intention to adopt PEMs within software development processes. In the remainder of the text, when we say software developers, we imply software architects, software designers, software engineers, and all other information technology professionals who are involved in designing and developing software systems.

The study conducted here is a remote survey with 149 software developers. Through a remote web application, we first guided the participants to use a PEM that enables them to understand and measure data privacy risks in an application design task. Participants were guided in a

step-by-step process to use the PEM and see the outcome of the steps they followed. Then, through a questionnaire developed based on TAM constructs, we asked the participants to evaluate the PEM they used on their intention to use it when they implement software systems. Our findings identify the factors organisations need to consider when deploying PEMs at their development processes.

The rest of the article is structured as follows: We first discuss recently proposed PEMs and the ways those PEMs have been evaluated. Next, we discuss the methodology and the experiment we conducted to evaluate software developers' intention to follow PEMs. Then, we present our findings with the discussion of the findings. The article concludes with the limitations of the experiment and conclusions.

2 RELATED WORK

2.1 Privacy Engineering Methodologies

Spiekermann and Cranor [67] define privacy engineering as a systematic effort to embed privacy concerns into the design of an application. It is an emerging area of research that focuses on facilitating software developers to embed privacy into the software systems they design (Privacy Engineering ISO standards: ISO/IEC PDTR 27550).¹ Martin and Alamo [52] identify four key forms of privacy engineering: (1) techniques or procedures that accomplish engineering tasks related to privacy, (2) methods that provide guidance or rules for achieving a set of privacy goals, (3) theories that deal with privacy, and (4) tools that support a developer to carry out engineering tasks related to privacy. In this research, we focus on the evaluation of the intention of developers to use the second form of privacy engineering, i.e., methods that provide guidance or rules for achieving a set of privacy goals. In this article, we call these PEMs.

2.2 Evaluating Privacy Engineering Methodologies

Different research has adopted different means to propose and evaluate PEMs. Table 1 presents a summary of recent PEM implementations from literature. From various PEMs that have been proposed and established, research that consists of an evaluation component for the PEMs was first chosen. Among them, research that used different techniques for PEM evaluation was then selected to get an overview of the current approaches that are taken to evaluate PEMs.

Oetzel et al. [59] and Wuyts et al. [77] both conducted evaluations of the utility of PEMs. Their evaluations focused on how competent or skilled the proposed PEM is in achieving its intended goal of enhancing privacy in a system design. Similarly, Li et al. [48] conducted a lab-based control group method to evaluate the usability and effectiveness of the PEMs. LINDDUN [77] is an extensive methodology that requires a considerable amount of work with a large set of parameters to be taken into account in the process [50]. It was tested in a longitudinal study with several workshops where participants engaged with the process for several days.

Authors of all PEMs have taken measures to theoretically and experimentally prove that the PEMs are effective and useful in instructing developers to embed privacy in software systems [36, 50, 77] or to show that the proposed PEMs are better than their counterparts [12, 34]. Above evaluations extensively consider the functionality of the PEM. However, regardless of the capability, competency, and effectiveness of a PEM in embedding privacy into system designs, previous research on developers' acceptance of methodologies at work suggests that PEMs could still fail in the development process [8]. Therefore, while the above studies are important to demonstrate the functionality of the PEMs and how effective they are in addressing the privacy requirements in system designs, whether developers would be willing to use these methods at work and whether

¹<https://www.iso.org/standard/72024.html>.

Table 1. Existing PEMs and Their Evaluations

PEM	Research Focus	Evaluation
Privacy Impact Assessment Methodology. Oetzel et al. [59]	The PEM focus on providing an approach for system designers to conduct a systematic privacy impact assessment. Provides step-by-step guidance to evaluate and address privacy impact to different stakeholders of a system.	Evaluation focused on evaluating the utility of the PEM in a very broad sense. They conducted a field experiment (qualitative) using action research (workshops) in a software development organisation. Utility of the methodology gives a measurement on how capable the methodology is in achieving its intended goal. Results showed that participants accepted the utility of the methodology.
Supportive tool for privacy. Li et al. [48]	Annotation-based supportive tool to encourage privacy decisions during system development activities. The tool requires developers to identify data used in a system design and annotate how the data item is used in the system, encouraging privacy protection at implementation level.	They conducted a qualitative evaluation using the control group method in lab conditions to test the tool's usability and effectiveness. Their results showed that the group that used the tool used only coarse-grained data in the system and used private storage for sensitive data encouraged from the annotations.
Privacy threat modelling framework (LINDDUN). Wuyts et al. [77]	A systematic framework to model privacy threats in a system and address them.	Conducted an extensive evaluation using master's students in the university. Three separate studies with students enrolled in a master's course on the correctness, completeness, productivity, ease of use, and reliability of their model to identify privacy threats in software systems, where participants engaged with the PEM for several days, used it for threat identification in system designs, and then discussed and evaluated the model. Their method provided good results for their methodology using mainly quantitative metrics on productivity and correctness.
PRIS, Privacy Requirements In System Designing [36]	A requirement analysis tool that focuses specifically on privacy requirements of a system in the early stages of a system design. Provides a set of concepts for modelling privacy requirements and translates these requirements into system models for implementation.	Applicability of the tool is demonstrated through real case studies. No empirical evaluation with stakeholders were observed.
QTMM, Quantitative Threat Modelling Methodology. Luna et al. [50]	A quantitative threat modelling methodology to draw objective conclusions about different privacy-related attacks that might compromise a system. Provides tools to support decision making to address privacy threats with functional system requirements and the corresponding trade-off before developing a system.	The use of the methodology is shown by applying the methodology in a real case study of a university course evaluation system, analysing its findings against similar threat modelling frameworks such as LINDDUN.

(Continued)

Table 1. Continued

PEM	Research Focus	Evaluation
Propan, Problem-Based Privacy Analysis. Beckers et al. [12]	A privacy threat identification methodology based on problem frames (a means to describe software development problems).	No empirical evaluation; applicability is demonstrated through a real use case of a health record system.
STRAP, a structured analysis framework for privacy. Jensen et al. [34]	A procedure to enable designers to evaluate a system on its privacy requirements and then provide solutions to the identified requirements.	A comparative analysis against its base method [13] was conducted on an existing software system with 32 college students. The goal was to show STRAP performs better than the base method by Reference [13]. Results showed that STRAP is more effective compared to the standard Bellotti and Sellen heuristics [13], requiring as much time on task yet resulting in more privacy-related vulnerabilities discovered.

developers perceive and understand the theoretically proven benefits of these methods is equally important to ensure the actual use of the PEM. However, the actual deployment of the PEM and its use in a system development process have not been investigated so far.

Most PEMs depend on the top-down approach for its acceptance in the development community. That is, PEMs are generally expected to be enforced by the organisation and then be used by developers. This is emphasised by research that reveals that the adoption of privacy in software development by developers is significantly affected by the organisational culture and support by the management [28, 33, 65]. However, it is presumptuous to expect that once organisational mandate is enforced, a PEM would be accepted by the software developers without any resistance. It has been found that if a methodology does not work well with the way developers work and if developers do not find a methodology useful, it is likely to be unsuccessful [8, 30]. Therefore, evaluating developers' intention to use PEMs is an important measurement to ensure whether developers would really use the guidelines in their software development activities.

2.3 Evaluation of Software Development Methodologies

PEMs are a special category of software development methodologies. Evaluation of the use of software methodologies in a development process in general is a relatively mature area in information systems research. The field of software development is a rapidly changing environment in which new tools and methodologies for process improvements are constantly proposed, tested, and used. Different researchers have adopted different techniques to evaluate the use and the intention to use or adopt different methodologies by software developers.

Hardgrave and Johnson [31] conducted an experiment to investigate the intention of developers to use an object oriented system development process. They determined that perceived organisational usefulness of the process is the only factor that directly affects a developers' acceptance of a process.

Khalifa and Verner [37] investigated developers' overall motivation for the use of software development methods. They found that in addition to the organisational support, developers' individual beliefs and the effect of the methodology on the quality of the software development process are important factors in motivating a developer to use a method. Riemenschneider et al. [61] showed that perceived usefulness, voluntariness, perceived compatibility, and subjective norm are significant determinants of software developers' intention to use methodologies at work. They studied

five different theoretical models: Theory of Planned Behavior (TPB), Perceived Characteristics of Innovating (PCI), TAM, TAM2 (an extended version of TAM), and the Model of Personal Computer Utilization (MPCU)) in their experiment to investigate developers' individual acceptance of methodologies. In a similar study, Sultan and Chan [68] reported that the characteristics of the methodology itself, such as its relative advantage and complexity, to be non-significant factors that affect the adoption of a methodology.

However, as highlighted in the introduction, these findings may not be directly applicable in the context of PEMs. Developers find privacy to be uncomfortable territory to them compared to other development requirements, because of vocabulary restrictions [28], unfamiliarity [27, 63, 67], and ambiguity [27, 67]. They also sometimes consider it to be irrelevant [44, 65]. This could result in developers being more resistant to using PEMs. Therefore, this study attempts to identify the factors that affect software developers' intention to use PEMs in their development practices.

2.4 Evaluating Intention to Use

In general, users' intention to use a methodology is influenced by the usefulness of a methodology in enhancing the quality of work or performance [20, 35, 55, 72]. Hardgrave et al. have shown that this remains true in the context of software developers [30]; that is, developers are likely to have a positive intention to use methodologies that they perceive to be useful. Similarly, developers demonstrate a positive intention to use less-complex methodologies (easy-to-follow methodologies), compared to complex and difficult methodologies [49, 59]. The compatibility of a methodology with the already established processes is also a significant construct on the intention users have to follow the methodology [30, 71, 73]. It is said that developers also try to bypass or reject methods that are not compatible with the way they already work [8]. Similarly in software development environments, where team work is important and common, the pressure from colleagues is said to be an important factor that influences the intention developers have to follow methodologies at work [8, 28, 30]. If their peers demonstrate positive intention to follow a methodology at work, then developers are likely to be influenced to use the methodology themselves. The ability of the methodology to demonstrate the results of using it is another factor that affects users to have an intention to follow the methodology. If developers are unable to see what they can achieve by using a methodology, then they find it difficult to use the methodology [18, 63]. For example, if a methodology clearly demonstrates the results of using it, users are likely to have a positive intention to use the methodology [73]. Following this existing knowledge, in this study, we hypothesise a model that would enable us to understand software developers' intention to use PEMs within software development practices.

Among many methodologies and theories used, we observed that TAM [19] is widely used in investigating the intention of users to adopt Information Technology solutions. TAM is a model used to evaluate the acceptance of technology by users (in our case, software developers) [19, 73]. For example, TAM is used to understand how teachers adopt technical tools to support their teaching methods, or how students adopt tools for studying, and so on [46]. There are various versions of TAM modified from the original model to use parameters that identify the aspects that affect technology acceptance by different user groups [73]. For example, the TAM used to evaluate technology acceptance by students would not be suitable to evaluate the technology acceptance by teachers. Previous studies have used TAM to evaluate the acceptance of software tools by software developers [22]. Hardgrave et al. [30] have proposed a TAM that is specifically designed to identify the factors that affect software developers to accept methodologies at work. They have shown usefulness, compatibility, social pressure, and organisational mandate to be direct determinants of developers' intention to use methodologies.

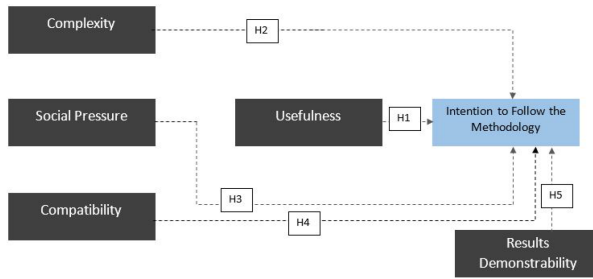


Fig. 1. Hypothesised evaluation model based on the TAM model proposed by Hardgrave et al. [30] with the additional construct of result demonstrability from Reference [73]. The dark blue items are the dependent constructs, and the light blue item indicates the independent construct. Dashed arrows indicate hypothesised relationships based on previous literature.

3 RESEARCH APPROACH

The model proposed in this article (Figure 1) hypothesises the following constructs as the independent variables that would affect software developers' behavioural intention to follow a privacy engineering model for embedding privacy into a software system:

- Usefulness of the model proposed (for job performance),
- Complexity of the model,
- Social pressure to use the model (influence to use), and
- The compatibility of the model with the way developers currently work
- Results Demonstrability.

The dependent variable in the model is, therefore, Software developers' behavioural intention to follow a PEM.

3.1 Constructs and Hypothesis

3.1.1 Perceived Usefulness. Hardgrave et al. [30] define “usefulness” as the degree to which a developer expects that following a methodology will improve his or her individual job performance. In the context of our research, we define usefulness as the degree to which the proposed model would help developers to embed privacy into a software system they develop. It is said that developers are likely to follow methods they perceive to be useful for their job performance, as better performance at work would result in promotions and benefits. At the same time, useful tools would enhance the quality of their work, resulting in personal satisfaction of their performance [20]. Previous studies have used different constructs to measure the usefulness of methodologies within software development practices. Moore and Benbasat [55] measured relative advantage of a methodology, which is the same as the usefulness measured in TAM. When it comes to tools, the usefulness is accepted to be a strong construct that influences its adoption by users [72]. In a methodology context, Johnson et al. [35] used process usefulness, product usefulness, communication usefulness, and career usefulness when eliciting beliefs about Object Oriented (OO) development. Similarly, Khalifa and Verner [37] examined process quality and product quality, which when improved would improve the usefulness as determinants of prototyping and waterfall usage.

In the context of privacy methodologies, previous research has identified that developers find existing privacy theories and definitions difficult to use, as they cannot perceive how the theories would be useful to what they do when they develop software systems [63]. Therefore, it is important that a privacy engineering methodology is perceived to be useful by developers. In the

adoption of any methodology, if the user is convinced that the methodology would improve the quality and performance at work, it is likely that the user would have a positive behavioural intention to follow the methodology. Therefore, we hypothesise that if a privacy engineering model can convince the developer that the model is useful for their performance at work in embedding privacy into the systems they develop, it is likely that the developers would have a behavioural intention to use it. Based on this, we form the first hypothesis,

H1: Usefulness of the privacy engineering methodology would positively affect the developers' behavioural intention to use the method when they implement privacy in software systems.

3.1.2 Perceived Complexity. It is said that complex methods that are difficult to follow are not likely to be used by developers. Complexity refers to the degree to which a software developer regards a methodology as difficult to follow or learn. Less complexity in a methodology results in increased efficacy in a user. If a method is too difficult to use, then it is less likely that the user would accept the methodology. Software developers are working in rapid environments at a fast pace. They are expected to execute tasks fast. In such an environment the complexity of a methodology would be an important determinant that affects the developers' intention to accept the methodology.

Especially in the context of security and privacy implementations, which is usually considered as a non-functional or secondary requirement in software application development by developers [4, 10, 57, 63], developers are less likely to follow a complex methodology that would take a significant amount of time and effort to learn and execute. Previous research focusing on software developers' privacy engagement has identified that privacy methodologies that communicate complex privacy requirements may not be successful in guiding developers, as developers lack comprehensive knowledge on privacy [63]. For example, it is said that when developers are required to make soft decisions based on ambiguous privacy requirements, they find it difficult to follow methods that guide them to embed privacy into the systems they design [59]. Loser and Dageline [49] found that in modern development processes such as agile development where regular priority decisions are made, more complex security requirements or requirements that need a higher effort were skipped with various excuses and strategies to defer the task. Therefore, in modern software developments, privacy methodologies should be simple and straightforward. Based on this, the second hypothesis we form in this research is,

H2: Complexity of a privacy engineering methodology would negatively affect the developers' behavioural intention to use the method in implementing privacy in a software system.

3.1.3 Perceived Social Pressure (Subjective Norm). This construct of social pressure, or subjective norm, was introduced to TAM by Venkatesh et al. [73]. It is the attitudes and perception of peers and management in the work environment [30]. In development environments developers work in teams, and therefore software development is a highly social process [45]. Because of this, the influence of peers and mentors has a significant effect on a developer's intention to follow a methodology at work [28]. Even if he himself does not find a methodology useful, he may still be inclined to use it at work if everyone else in his team considers it to be important. Especially when it comes to embedding privacy into a software system, it has been identified that developers consider privacy to be an informal aspect of software development that is more dependent on the culture that is developed within the organisation than the policies or guidelines that are in place for them to follow [8]; that is, developers are likely to be motivated through the attitudes and perspectives of their peers and management rather than the policies that enforce them to embed privacy into a system design [8].

However, in the context of privacy, developers are found to have significantly different attitudes towards their responsibility in ensuring end-user privacy. Previous studies have shown that developers consider privacy to not be an immediate threat and not their responsibility. Therefore, we are skeptical about the influence of the impact of subjective norm on the behavioural intention of a software developer to use a methodology that guides him to embed privacy into software systems. Nevertheless, to empirically investigate the influence, we form the third hypothesis in this research as,

H3: Social pressure has a positive effect on software developers' behavioural intention to use this method for embedding privacy into the software systems they develop.

3.1.4 Perceived Compatibility. Tornatzky and Klain define compatibility as “congruence with the existing practices of the adopters” [71]. That is how consistent a proposed methodology is to the existing practices of a developer. Working in highly dynamic, challenging environments, developers do not appreciate distractions from their usual work patterns. Much research has been conducted to investigate the impact of distractions, compatibility issues with new methodologies, and ways to minimise the incompatibilities at software development environments. Therefore, being compatible with the existing workflows and patterns is an important construct that affects software developers' intention to follow a methodology at work.

Especially in the context of privacy methodologies, previous research has identified that software developers tend to disregard privacy guidelines or methods that significantly deviate from the existing software development models or practices that are already established [8]. At the same time, it has been identified that developers do not prefer breaking/modifying an existing client/server model to embed privacy into a system, even if it guarantees privacy [8]. This suggests that developers are likely to reject privacy models that are incompatible with their existing work patterns and coding patterns. If privacy engineering methods are to be adopted, then proposing completely new methodologies from scratch that are disconnected from existing development practice may not be succeed. Rather, privacy engineering needs to be aligned with existing development practices [52]. Based on this, we form our fourth hypothesis in this study,

H4: Compatibility of the privacy model with the existing practices of the developer has a positive effect on the intention of a developer to use the model.

3.1.5 Perceived Result Demonstrability. Upon looking for specific constructs that could affect software developers' intention to follow privacy engineering methodologies, we identified that the lack of clearly defined results in following privacy methodologies has resulted in software developers not following theoretical privacy guidelines at development environments. For example, it has been shown that developers have said that the privacy methodologies are not convincing and do not provide the user the confidence that the methodology would improve the privacy in an application, and hence they are not sure if and when they should follow privacy methodologies at work. Therefore, previous research on privacy has showed that developers need to see how a privacy engineering method would improve the system they develop to follow it [63]. Need for demonstration or assurance of privacy has been identified as a requirement in establishing privacy guidelines by the ENISA guidelines for Privacy and Data Protection by Design [18]. If a methodology does not show how its use would result in implementing better privacy in a software system, then developers would not be convinced to follow the methodology.

The capability to identify how a proposed methodology would help them to improve their job performance or the quality of their work is therefore identified to affect a person's intention to follow a methodology [20]. While Hardgrave's TAM model for software developers' intention to follow a methodology does not identify this, Venkatesh et al. [73] have identified that the

“tangibility of the results of using the innovation” [55] has an influence on a user’s intention to make use of the innovation. Based on this, we propose the fifth hypothesis.

H5: Result demonstrability would have a positive effect on software developers’ intention to follow a privacy engineering methodology.

3.1.6 Intention to Use the Methodology. Finally, intention to use the methodology when implementing data protection in a software system is the dependent variable in this research. This construct of intention to use defines how willing a user would be to use a methodology or how much effort a user would be willing to exert to perform a certain behaviour. In the context of this research, intention to use the methodology means the willingness of a developer to use the proposed model when they are asked to implement data protection in a software system. Our research observes how the independent constructs introduced above would affect the intention of software developers to use a privacy engineering method when they embed privacy into a software system.

4 METHODOLOGY

4.1 Participant Recruitment

We recruited participants by emailing GitHub users who are members of popular repositories that use web application development technologies. We queried the 100 most popular NodeJS, JavaScript, PHP, AngularJS, React, and Vue repositories from October 2018 to December 2018. This approach is used for recruiting participants for surveys that target experienced software developers due to the difficulty in recruiting developers from diverse backgrounds [75], whereas lab studies would only allow us to recruit participants from a specific geographical location.

This approach allowed us to conduct a remote experiment. Remote studies encourage participants to participate in the study during their free time in their natural environment (home/workplace) [5, 75]. This allows them to behave close to how they usually behave, whereas lab studies may encourage participants to demonstrate behaviours different to what they normally do due to being monitored.

We sent an invitation email to the invitees with the link to the survey, and participation was voluntary. This invitation email provided participants with information about the study, the type of data we collect, a brief summary of the tasks they have to do in the survey, and the estimated time spent on the survey. Participants were required to be above 18 years of age, and participants were expected to have some development experience.

4.2 Study Procedure

The study involved a task to use a PEM in a software design activity and to answer a questionnaire developed based on the theoretical model proposed.

The study used a simple, straightforward PEM that guides software developers to implement data protection in the software systems they develop through understanding data attributes such as data sensitivity, data visibility in the system, and the relevance of the data to the system [64]. This PEM is a step-by-step approach that enables developers to understand and consider those attributes of data and measure the privacy risks associated with the data. The PEM, therefore, guides developers to implement measures to protect data based on the privacy risk of the data. An outline of the PEM is available in Appendix H Figures 5 and 6. We implemented this PEM in a simple web application so developers could follow the PEM through a task. Our goal in this evaluation is to evaluate the behavioural intention of developers to use this PEM.

We designed two or three questions to represent each construct. We first adopted questions for each construct from previous studies that investigate software developers’ behavioural intention to follow a methodology when they develop software. We adopted the questions for Complexity

from [20], Usefulness and Compatibility questions from References [69] and [20], Social pressure questions from Reference [73], result demonstrability questions from Reference [73], and the behavioural intention questions from Reference [30].

The survey was hosted on our own servers. We built our own survey web app for two reasons. First, we wanted to give participants feedback based on the choices they made in the survey. That is, based on the way they designed the system for data items, we wanted to calculate the data privacy risk and present them with the values. This is not possible in most freely available survey platforms such as Google Forms. Second, during our previous studies with developers, we have observed that surveys hosted on third-party servers sometimes make participants lose trust in the survey, leading to reduced participation. Especially on security- and privacy-related surveys, developers get back to us saying they are not comfortable filling in a Google form knowing it may expose their email address as a participant. Therefore, we developed a simple JavaScript/JQuery and HTML web application and hosted it on our own servers. This way, we could assure the participants that we only collect the data that they entered in the survey.

The survey used an application scenario as the context for the developers to use the PEM. We used a health data related app as the context to provide participants with a scenario where they are tempted to collect user data that would be profitable if used within the law, and they have to be careful because the data is sensitive to the user. The application scenario description provided to the participants is available in Appendix A.

On the first page of the survey, participants were asked to proceed to the survey if they gave us consent to access, store, and use their answers for our research. We gave details about us (the research team) and information about the steps they have to follow on the first page of the survey. We also provided them with the ethic approval grant codes and contacts to reach out if they come across any concerns while completing the survey. The list below shows the steps the participants (developers) had to follow in the survey:

- Step I: Read and Understand the application scenario (available in Appendix A).
- Step II : Select the data to be collected for the system (a pre-defined list of data items were provided, and participants could add their own data items if they wanted).
- Step III: Decide on the storage and sharing of the selected data in the system.
- Step IV: Follow the privacy methodology - Determine the sensitivity, visibility in the app and the relevance of each and every data item on a scale of three levels. The definitions of the terms (sensitivity, visibility, relevance) and the description of the scales for categorisation were provided (refer to Appendix C).
- Step V: Follow the privacy methodology - Observe the system-calculated privacy risk values based on the sensitivity, visibility, and relevance values selected. The formula that calculates the privacy risk was provided with a description to understand how the risk value was derived.
- Step VI: Follow the privacy methodology - Change the decision to collect data, and store and share data based on the risk values. (Participants were told that removing irrelevant data and limiting data visibility would reduce the privacy risk.)
- Step VII: Evaluate the privacy methodology followed based on the questions following the TAM model (available in Appendix B).
- Step VIII: Provide basic demographics (age group, experience, gender, and profession).

The web app guided participants along each step. Participants were provided with descriptive instructions to follow each step and were encouraged to get back to us with any concerns or feedback.

4.3 Pilot Study

A pilot study was first conducted to evaluate the validity of the research approach. It is a usual practice to conduct a pilot study with a relatively small sample size and make the required adjustments to the questions and the study procedure based on the problems identified. First the questionnaire was designed and the pilot study was conducted with 10 participants.

The first questionnaire used four complexity questions, three usefulness questions, two social pressure questions, three compatibility questions, two organisational mandate questions, two result demonstrability questions, and two behavioural intention-to-use questions.

We first conducted two face-to-face evaluations of the questions with two developers known to the researchers who were not involved in the research. We changed the wording of the questions and added explanations where the participant developers said the meaning of the questions was not clear to them.

To recruit participants for the pilot study, 6,552 invitation emails were sent (Github users) from the 10th to 18th of October 2018; 85 of these emails bounced back due to non-existent or fake email addresses, and we received 38 submissions.

4.3.1 Outcomes of the Pilot Study. Results demonstrability, compatibility, and complexity statements received answers with low consistency (Cronbach's $\alpha < 0.6$, a Cronbach's $\alpha > 0.7$ is considered acceptable [58]). When looking at the compatibility and complexity statements, we observed that they were relatively long. Therefore, we replaced the longer statements with shorter ones, removing unnecessary words that are obvious. For example, the statement "Following these steps in a software development process is clear and easy for me" was shortened to "Using these steps is clear and easy."

In the social pressure questions, we observed that one statement had a very different mean value compared to the other two statements. That statement said, "People who influence my work would support using this privacy risk value at work." We understood that the wording "people who influence me" could imply managers who instruct developers. Pressure from a manager could be considered as an order or a mandatory action, which is different from peer pressure. Therefore, we removed that statement and only considered the peers of a developer in measuring the social pressure.

Then, we also added negation questions to each construct so we could identify participants who do not answer the questionnaire with genuine commitment. That is, in the case of Likert scales, it is possible that participants answer the questions with the same option after reading one or two statements, expecting the questions to be similarly structured. When we use the negations of the same statements in the questionnaire, we can identify such submissions and remove them from the results being analysed. The final set of questions that was used in the main study are available in the Appendix B.

4.4 Main Study

4.4.1 Participants. 15,078 invitation emails were sent in total from the 23rd of October to the 1st of December 2018; 89 of the invitation emails bounced back due to non-existent or fake email addresses. We received 168 (0.0111 participant rate) submissions for the study, out of which 19 were eliminated during data analysis.

We eliminated invalid submissions using two approaches. All submissions that had the same answer for all TAM questions were eliminated; that is, if a participant had clicked the same answer in the Likert scale for all the questions, those submissions were removed. Next, based on the negation questions, we eliminated submissions that were not genuine. That is, if a participant (strongly) agreed with a statement, s/he should (strongly) disagree with its negative statement, otherwise it

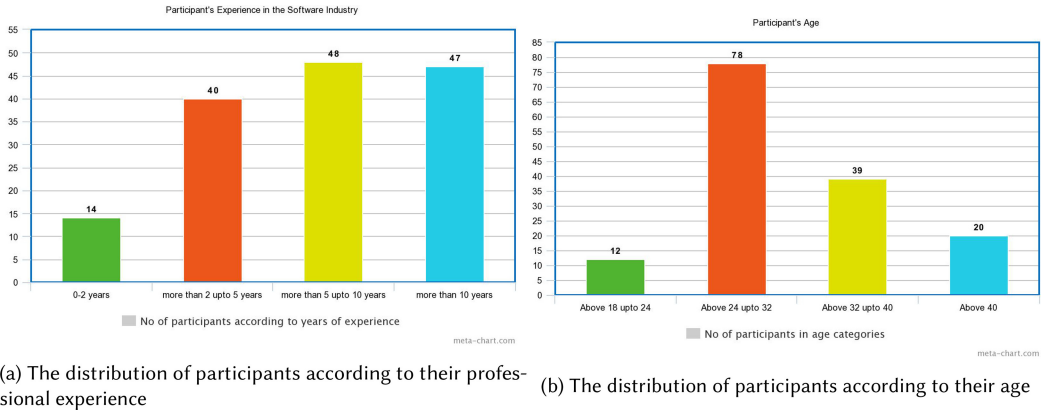


Fig. 2. Participant demographics - I.

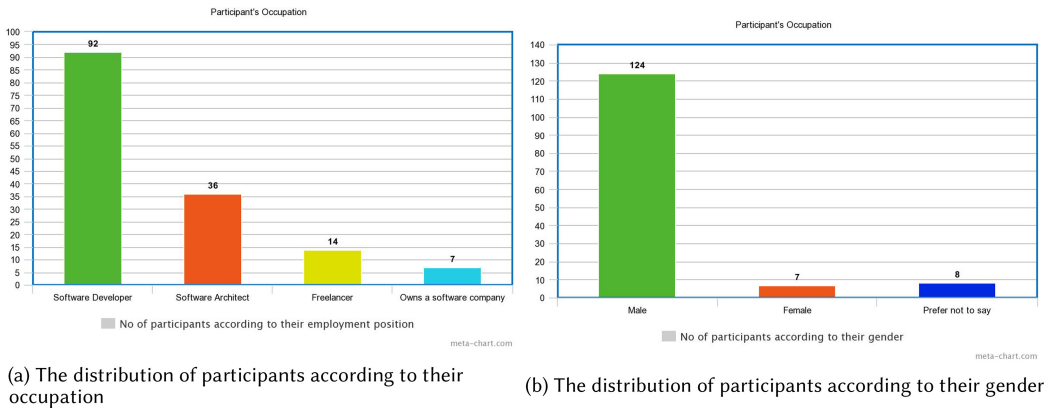


Fig. 3. Participant demographics - II.

is highly likely that the particular participant randomly answered the questions without reading them. Altogether, 19 answers were eliminated out of the 168 submissions we had, and we were left with 149 submissions.

Participants for this experiment were recruited using the GitHub social coding community to reach the software developers that would represent the population of developers well enough, following several previous studies [24]. GitHub has over “30 million visitors each month” [1] and is considered to be the largest social coding community in the world [26]. However, the low response rate (0.0111) and voluntary participation may suggest a biased sample of participants. The demographics presented in Figures 2(a) and (b) and 3(a) and (b) show that the age range, occupation, and years of experience of the participants are sufficiently diverse. Nevertheless, it can be observed that the sample we received was predominantly male (93%). However, as previous studies have shown similar gender percentages in survey respondents [26, 60], we believe this sample sufficiently captures the variations in the population of software developers. The demographic information presented here were collected through demographic questions asked in the survey (presented in Appendix I).

We also asked participants to let us know if they wish to hear the results of the study, and we got 48 emails expressing interest to receive the study results and information about the publication of the results.

4.5 Data Analysis

The goal of this study is to investigate the behavioural intention of software developers to use the PEM tested when they implement privacy in a software system. By the answers participants give in step 7, we aim to evaluate the behavioural intention of the participants to use the proposed method in their usual software design/development activities. All TAM questions were based on a five-point Likert scale from Strongly Agree, Somewhat Agree, Neutral, Somewhat Disagree, and Strongly Disagree.

We calculated the Kaiser-Meyer-Olkin (KMO) value of the data set after every round of invitations to measure participant adequacy. We stopped data collection when we reached a KMO above 0.9. The KMO of the final data set is 0.918. (A KMO value > 0.8 is considered good in calculating correlations [38].) Next, we calculated the consistency for each construct using Cronbach's alpha to make sure the data set is sufficiently consistent (the values are available in Appendix D). All constructors demonstrated a Cronbach's alpha value > 0.7 , which is considered reliable and adequate [17, 29, 40].

We evaluated the hypothesised model and derived a final model using Multiple Regression and Factor Analysis with Principle Axis Factoring. Principle Axis Factoring was used to identify the least number of factors, which can account for the common variance (correlation) among the set of variables used.

Then, the convergent validity of the constructs was calculated. For convergent validity, factor loading for all questions in the constructors were calculated (data is available in Appendix F). All factor loadings are > 0.6 (the lowest is 0.695). Therefore, the data set satisfies the required convergent validity [30]. Discriminant validity for the data set was also calculated (measurements available in Appendix E). All pairs demonstrated a significant chi-square value, and hence the constructs tested have sufficient discriminant validity [9, 80].

Once the data set for the analysis was finalised, the next task was to evaluate the model hypotheses (H1–H6). We used Structural Equation modelling for this analysis (SEM, a multivariate statistical analysis technique that is used to analyse structural relationships between one or more Independent Variables and one or more Dependent Variables). We defined direct relationships as expected using Hardgrave's model in AMOS and generated the model. We made adjustments as suggested by the modification indices in AMOS and arrived at the final model. This process is discussed in the results section in detail.

The model we finally derived is discussed at length in the results section. We used the SPSS and AMOS for all statistical measurements in this research.

5 RESULTS

All factor loadings were sufficiently large (Appendix G) demonstrating convergent validity [9, 11]. The chi-square values (Appendix E, Table 6) were observed to be significant for each construct demonstrating discriminant validity in all the constructs used in the model. Furthermore, all constructs were observed to have a reliability (Cronbach's alpha) greater than 0.7 (Appendix D, Table 5). Convergent validity of the determinants are given in Table 8, Appendix F.

By using SEM in SPSS it was seen that usefulness ($p < 0.001$), compatibility ($p < 0.005$), and demonstrability ($p < 0.005$) were significant determinants of the participants' intention to follow the tested PEM. Interestingly, Social pressure and Complexity had no direct significant relationship with intention. When modelling software developers' intention to follow methodologies, Hardgrave et al. [30] also found complexity to not have significant effect. Consistent with their findings, our model also suggests that complexity of a methodology does not have a direct effect. In addition to that, our study shows that social pressure does not have a significant effect on intention either.

Table 2. Model Parameters and Hypotheses

Construct	Beta	Sig.	Hypothesis
Usefulness	0.351	0.000	H1-accept
Complexity	-0.111	0.150	H2-reject
Social Pressure	-0.011	0.874	H3-reject
Compatibility	0.268	0.001	H4-accept
Results Demonstrability	0.265	0.001	H5-accept

Table 3. Model Fit

Construct	Expected acceptable-value [29]	Obtained values
GFI	$x > 0.9$	0.982
RMSEA	$0.05 \geq x$ good	0.047
AGFI	$x > 0.9$	0.925
NFI	$x > 0.9$	0.888 ** marginal
CFI	$x > 0.9$	0.918
CMIN/DFI	$x < 3$	2.843

Table 4. Descriptive Statistics of the Constructs
(5-Point Likert Scale)

Constructs	Mean	St. Deviation
Complexity (negation - ease of use)	3.554	0.816
Usefulness	3.637	1.04
Social Pressure	2.57	0.978
Compatibility	3.675	0.913
Demonstrability	3.785	0.834
Intention	3.980	0.940

Therefore, to observe any indirect effects, we evaluated indirect effect of both complexity and social pressure via the three factors that had direct effect (compatibility, demonstrability, and usefulness). Interestingly both social pressure ($p < 0.001$) and complexity ($p < 0.005$) had a significant direct effect on usefulness; however, with a lower model fit of 0.391. Then, to increase the model fit, we added more relationships into the model to measure other indirect effects that may not have been observed before. We added both compatibility and demonstrability. With an indirect effect for compatibility through usefulness, we obtained an increased model fit.

However, still we observed that CFI value in the model fit to be marginal (0.886). AMOS showed another interesting yet convincing modification index from demonstrability to social pressure. We find this relationship important, as this suggests that the result demonstrability of a PEM has a significant direct effect on social pressure. We therefore included this relationship in the model and tested the final model. The final model fit is presented in Table 3, which is considered a good fit [29]. The final model derived is depicted in Figure 4, and the results on the hypotheses tested is given in Table 2.

The resulting model of this study explains 58.5% of the developers' intention to follow PEMs. This is a comparatively acceptable value for a model that explains human behaviours [6]. For example, Venkatesh et al. [73] TAM model explained 40% of their results and Agarwal and Prasad presented a model for information technology acceptance that explained 46% of the results.

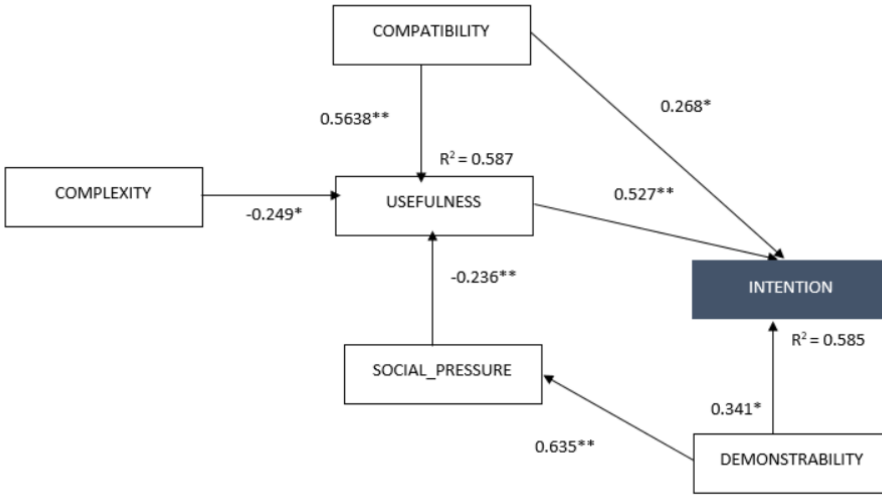


Fig. 4. The final model derived to measure the technology acceptance of software developers in adopting privacy engineering methodologies. The model shows that usefulness, compatibility, and demonstrability have significant direct effect on the intention to use, while social pressure and complexity have an indirect effect through usefulness. $p < 0.005^*$, $p < 0.001^{**}$.

The mean-scores for each construct and standard deviations are given in Table 4. The results demonstrate that developers had an intention to use the model we proposed in their application development tasks and they perceived the system to be useful, easy to use, compatible, and demonstrable. Participants showed positive intention to use the methodology (3.94 out of 5-point Likert scale). However, participants did not believe that their peers would accept the tool for embedding privacy into software systems. This is an interesting outcome, and we explore this further in the next section when we discuss the findings of the study.

6 DISCUSSION

Khalifa and Verner [37] identified that quality of development and team size are significant determinants of software developers' use of methodologies (not intention). Agarwal and Prasad [7] identified that developers' beliefs have a significant effect on their adoption of process innovations within software development environments. They showed that organisational tenure, prior technical knowledge, training experiences, and perceived job insecurity are factors that affect the beliefs formed by developers on process innovations introduced to them. Similarly, Sultan and Chan [68] investigated the effect of individual characteristics, group characteristics, company characteristics, and an individual's perception of technology on technology adoption in a software company. However, the current study takes a different approach from these studies. We focus on the intention of a software developer as an individual accepting a PEM at work, whereas Sultan and Chan and Agarwal and Prasad focus on adoption itself and not on intention. Their studies, therefore, do not provide insights to identify factors that affect developers' intention to use a methodology.

The experiment of Hardgrave et al. [30] investigated software developers' intention to follow software development methodologies at work. However, the study reported in this paper was conducted on the premise that PEMs are different from general software development methodologies due to several reasons. Privacy methodologies are said to be vague and complex [59, 63, 67]. Therefore, the use of PEMs may not be as straightforward as the use of general software engineering methodologies. At the same time, developers consider privacy to be a non-functional requirement

in system development [63]. Therefore, developers may not see the use of PEMs as important as the use of other software development methodologies. Consistent with these assumptions, our findings show that developers' behavioural intention to follow PEMs are quite different from Hardgrave's findings.

First, the effect of usefulness, while being the highest among other factors in the model, was still low ($\beta = 0.527$) compared to similar studies. Most previous studies have shown a higher effect of usefulness on adopting methodologies. Explaining a usefulness effect of 0.46, Hardgrave et al. suggest [30] that usefulness of methodologies in software development practices is better realised from an organisation's perspective rather than from an individual developer's perspective. We believe that the usefulness of PEMs is realised from the resulting privacy in the software systems, in terms of protecting the reputation of the organisation and financial liability in the event of a privacy breach. Unlike other features in a system, such as the execution speed of a program or UI quality, usefulness from an organisational perspective may not be directly apparent to a software developer. At the same time, usefulness of a PEM takes time to be realised. That is, privacy of the software would be realised once the software is developed and used by users, or once the application is evaluated by privacy experts. However, in our study, participants answered the questions after interacting with the methodology in a development activity and, therefore, it is possible that the methodology did not communicate its usefulness as such in this single engagement. This may have resulted in the low effect of usefulness observed in the intention of developers to use the PEM. In spite of this, previous research has shown that communicating the benefits of an action to a developer is effective in inducing positive motivation in developers to take those actions [21]. Therefore, one cannot completely disregard the effect of the usefulness of a PEM towards the intention of the developers to use the PEM. Perhaps a PEM could encourage developers to use the PEM by showing how the developer could gain skills through using the PEM and how it enhances their knowledge on data privacy in general. Showing how the PEM helps the organisation in terms of business and reputation would be one way of showing the usefulness of a PEM from an organisational perspective to the developer.

Second, the study demonstrated that social pressure had no direct effect on developers' intention to use PEMs. The indirect effect it had on the usefulness was also found to be negative and small. This is an interesting finding. In work environments that are heavily team based, similar to software development, social pressure is assumed to be a powerful determinant of the workers' intention to use methodologies [30, 35]. It is said that individuals may decide to perform actions even if they do not believe strongly in the consequences, based on the reactions and interests of their peers. However, our findings show that the effect of peers at work (social pressure) on the usefulness of the PEM is negative. That is, even though the participants themselves had a positive intention to follow the methodology (3.98 out of 1–5 Likert scale), they considered their peers to have a negative attitude towards the PEM (the mean value for social pressure was 2.5 out of a 1–5 point Likert scale). It is interesting that the participants perceived their peers to be not interested in a methodology they had a positive intention to follow at work. The model explained this negative effect through the usefulness of the PEM suggesting that developers express themselves negatively on the usefulness of PEMs among peers. Interestingly, we observed that other experiments on methodology acceptance and tool acceptance have shown positive effects from social pressure (strong, weak/direct, indirect) [30, 73]. The negative effect observed here may suggest that developers tend to disregard their peers' opinions when they perceive the PEM to be useful, compatible, and demonstrative. It could equally imply that developers misinterpret their peers' opinions towards PEMs probably due to the way they express their interest towards PEMs among peers. It could also mean that developers do not discuss their opinions on PEMs because they do not believe that would have an impact on their peers. For example, qualitatively investigating peer

recommendations and peer observations for tool adoption among developers, Emerson and Gail [56] identified that developers rarely discover new developer tools from their peers. Xiao et al. [78] have also shown that peer recommendations for security tools do not happen often because “e developers do not often drop by other developers’ offices [to recommend tools].” One could safely argue the same in the context of privacy methodologies. However, we believe that further studies to observe the effect of social pressure in the context of PEMs would provide better insights for explaining this finding.

Third, we found complexity to have a very small indirect effect on intention. This is consistent with the findings of Hardgrave et al. [30]. Based on this finding, one could argue that developers’ intention to follow a methodology is not affected by its complexity. Nevertheless, natural instincts suggest that developers would find complex, hard-to-use methodologies difficult to follow and hence would have a lower intention to use them at work. For example, it is found that stress, anxiety, and burnout induce unhappiness, which results in low motivation and less performance in software developers [26]. Similar studies that explored intention to use software methodologies in general have shown complexity as a direct significant contributor to intention to use [20, 53, 70, 73]. Similar to ours, Hardgrave et al. [30] have shown that complexity has a negative indirect effect on software developers’ intention to follow methodologies. Comparing the difference of their results to previous studies, Hardgrave et al. have argued that their indirect effect is due to an efficacy complexity, which identifies developers’ lack of confidence in their ability to follow the methodology rather than the complexity of the methodology itself. The PEM used in the experiment had a very low perceived complexity (ease of use is 3.55 out of 5-point Likert scale). Therefore, developers may have not felt challenging difficulties due to the method’s complexity that could have impacted their intention to follow the methodology. Future studies with different PEMs with different relative complexities could help interpret how the perceived complexity of a PEM affects developers’ intention to follow the PEM at work. It would also show how the difference in efficacy complexity as identified by Hardgrave et al. and method complexity (the inherent complexity to follow the methodology) affect the intention to use.

Next, our study showed demonstrability as a significant direct determinant that affects intention. Demonstrability has not been investigated in the context of software developers’ intention to follow methodologies before. Venkatesh et al. [73] introduced this parameter of perceived result demonstrability into TAM2 in determining intention to use. They suggest that if a system (in our case, the methodology) demonstrates readily discernible results to the user, users would have a positive intention to use that system. Previous studies have shown that result demonstrability is an important factor in using privacy methodologies. Zaltman et al. [81] suggested that the more “amenable to demonstration the innovation is...the more likely it is to be adopted.” Furthermore, demonstrability showed a direct significant effect on social pressure, suggesting that if a PEM can demonstrate its results it could influence social pressure on developers to use the PEM. Developers have expressed that the lack of demonstrability in privacy methodologies in general makes them difficult to follow [63]. Especially as privacy is not a straightforward concept in system development, if a methodology does not show how its use could improve the privacy of a system, it could confuse developers. Consistent with the previous studies, our findings showed that developers’ intention to use privacy methodologies is directly affected by result demonstrability.

7 IMPLICATIONS OF FINDINGS

Our study investigated factors that affect an individual developer’s behavioural intention independent from their organisational policies to adopt a PEM at work. In most organisational settings the adoption of a PEM within their software development environment is an organisational decision. The involvement of developers in this decision is minimal, and hence the use of the methodology at

Table 5. Total Effect of the Constructs Tested on Intention

Construct	Direct Effect	Indirect Effect	Total Effect
Compatibility	0.268	0.1976	0.4656
Usefulness	0.527	0	0.527
Demonstrability	0.341	− 0.052	0.2592
Social Pressure	0	− 0.0828	− 0.0828
Complexity	0	− 0.249	− 0.249

work is mostly enforced. However, this enforcement alone may not be effective in encouraging the developers to use the PEM at work. For example, Kotter [41] has said that organisational mandate alone is not sufficient in making developers accept a methodology. Further studies conducted in the IT field have consistently shown that organisational mandate is just another construct among many that affect developers' intention to adopt methodologies at work [30, 73]. If the developers are following these methodologies involuntarily, then it could lead to unhappiness and stress at work, which could result in developers bypassing the methodologies or manipulating the process for convenience. It is said that unhappiness in developers at work could lead to poor performance and quality of work, which could affect the organisation [25, 26].

This study focused on the attributes that could affect software developers' intention to follow PEMs other than the organisational mandate. The results, hence, would be important for organisations to see how they could implement their policies to ensure the PEMs proposed would be voluntarily accepted and used by the developers, and not just because it is enforced on them. In this section, we discuss the implications of the results to highlight how the results could be applied into the actual PEM deployment process within an organisation.

Table 5 shows the total effect (direct and indirect) of the measured constructs on the final intention of developers in order.

Usefulness of a PEM was shown to have the highest significant effect on developers' intention to accept PEMs. If developers do not perceive the PEM to be useful, then its success in practice could be affected. Introducing new methods could reduce productivity and raise issues in the short term [47], therefore, to convince developers it is imperative, explicit measures must be taken to communicate the usefulness of the PEM on the organisation and on the developers' personal skill development. This would induce intention in developers to use the methodology voluntarily until they could see the usefulness of the methodology for themselves.

Compatibility of the methodology is also shown to have a strong effect on developers' intention to use a PEM (second to only usefulness). Developers usually adhere to a work pattern that complements their performance and work output. If a PEM derails developers significantly from their usual work patterns, then it is unlikely that the PEM would have positive reaction from developers. Therefore, when designing a PEM, careful attention should be given to ensure that the steps of the PEM are compatible with the usual development practices software developers follow. Getting developers feedback prior to proposing and deploying the PEM and initiating team discussions on the appropriate ways to introduce the PEM to the current work routines may help successful deployment of a PEM that is compatible with existing work patterns of software developers.

Result demonstration of the PEM was shown to be a direct determinant of intention, with a very small indirect effect via social pressure and usefulness. The results of a PEM would be most visible to the organisation and users, and not the developers. Therefore, developers' intention to follow the methodology may be affected by the lack of tangible results for themselves at development time. Therefore, a PEM could take measures to show how each and every step of the PEM adds value to the end-product. Demonstrating the impact of not following PEMs could also be used

within the development environment to encourage developers to follow PEMs. This would help developers to realise the results of the steps they execute and, hence, encourage them to use the PEM.

The model shows complexity to have a small negative effect. This suggests that a PEM's complexity would affect negatively on the developers' intention to use it. Previous studies have shown complexity of a methodology to affect the use of tools and methodologies in the IT domain. Even though the effect is shown to be small, it suggests that PEMs should not be overly complex and difficult to use. A simple methodology would be less stressful to use and, hence, be effective in a software development environment.

This research uses the standard approach of TAM to determine the constructs that affect software developers' intention to follow PEMs. This could be used as a standard way to evaluate TAMs for its use by software developers. Consequently, organisations that develop standards and guidelines for PEMs could also benefit from these findings. For example, the ISO standards for Privacy Engineering (ISO/IEC PDTR 27550) is currently under development. The NIST standards for Privacy Engineering [14] is a widely accepted standard that specifies how privacy engineering methods should work in software development. Such initiatives could use the findings of this study to define guidelines for PEMs to ensure the use of PEMs by software developers.

8 LIMITATIONS AND FUTURE WORK

Using a single PEM in a hypothetical setting through a remote study is a method limitation in this experiment. Ideal investigation of intention to use should be executed in an organisational setting with real development teams. This would help observing if the intention to use results in actual use, and if prolonged exposure to the methodology being tested changes the opinion of the developers. However, this realism comes with the cost of lack of generalisability. Real-life case studies are specific to that particular organisation and therefore may not be interpreted to explain the general context. Therefore, the research approach adopted by most researchers is achieving mundane reality [66], which refers to the resemblance of the experiment conducted in real-world scenarios in terms of tasks, participants, and the behaviour of the participants. The study therefore attempted to achieve mundane reality.

Remote execution helped us achieve a close-to-realistic environment than a lab setting. Existing work suggests that if special attention is given to designing the task, remote studies could be used to investigate the behaviours of participants in tasks ranging from simple surveys to prototype development [3, 39]. In this study, the participants were assigned with a software application design task (study task) and the survey was given as a post-task questionnaire. This approach was used to minimise the limitations in survey methodologies that arise due to participants' memorability and recall capacity of their usual actions when they answer questions [4]. By asking questions after the participants completed a designated activity, they are asked about their general behaviour. Rather, they are inquired about the task that they have completed. Therefore, rather than expressing what they think they would do, or what they think they did in the past, participants express what they did in the task. This way the experiment attempts to identify the real experience of the participants. Therefore, we believe that the remote-task-based approach had minimal effect on the validity of the results [3, 39].

Also, by recruiting real, experienced developers crowd-sourced through GitHub, we tried to achieve a realistic participant pool [3]. Nevertheless, the opt-in rate for the invites was low (0.0111 participant rate) and one could argue that those who participated in the study had an inherent interest towards software privacy and privacy guidelines in general. Research has shown that volunteers in research experiments may not deviate from the general population in terms of behavioural characteristics [62]. Acar et al. [5] also observed that the GitHub participants who opted in to a

study invitation were relatively more active in GitHub than those who did not opt in. Therefore, the participants in this study may not be a representative sample of the GitHub population. It is even less representative of the general population of software developers.

As the design task mentioned data privacy, participants may have gotten the idea that the experiment was about data privacy. This may have interested those who have an interest towards data privacy and discouraged those with a prejudice against data privacy (it is known that some developers do not consider data privacy as their responsibility [28, 65]). This may impose some threats to validity of the results. However, this threat is controlled to some extent, as the participants were not aware of the exact constructs/hypotheses that were tested and the expected results of the experiment. Participants were not informed that they were following a PEM and they were not informed that their intention to use the PEM was being evaluated. The PEM was identified as steps/method in the questionnaire for this purpose.

It should also be noted that the intention to use identified here may not explain the actual use of the method. Longitudinal studies in real organisational settings would help observe how the intention to use identified here explains the actual use of PEMs in developers when developers are exposed to the PEM for a prolonged period. These findings, therefore, should be taken as guidance when deploying PEMs within organisational settings to minimise the initial resistance from developers.

We therefore accept that there exist some threats to internal validity (voluntary participation, research instrumentation) and external validity (GitHub sample) of the results drawn in this research. Ideal results that are generalisable without limitations are not realistic. Therefore, if the results are interpreted with an understanding of these limitations, within the limitations of these threats, the results are valid.

9 CONCLUSION

This research investigated developers' behavioural intention to use PEMs in software development activities. Using the Technology Acceptance Model, the study investigated the effect of five constructs on software developers' intention to follow PEMs. The findings reveal the usefulness of a PEM to be the strongest contributor to a developer's behavioural intention to follow a privacy engineering methodology, followed by compatibility and demonstrability of results. Perceived complexity was shown to have a small negative effect consistent with similar studies. However, social pressure was interestingly found to be negative and small, which was different from previously observed results on developers' intention to follow methodologies at work. Findings of this study could be used to enhance the current knowledge of using PEMs within software development practices and also to understand developers' perception of PEMs. Results would help organisations that expect to deploy PEMs within their organisations to enhance the effectiveness of PEMs being deployed within software development environments.

APPENDIX

A APPLICATION SCENARIO USED IN THE EXPERIMENTS

A web-based health-care application that allows remote consultation with medical professionals, general practitioners and specialists, for a payment. Users should be able to browse through a registered list of medical professionals and chat (text/video) with them on their health problems for advice. Doctors and health-care professionals can register on the application to earn by providing their expertise to users. The application is to be freely available on-line (desktop/mobile).

B TAM STATEMENTS USED IN THE STUDY

Complexity

- CMPLX1: Using these steps is clear and easy.
- CMPLX2: I already have the required knowledge to use these steps to determine privacy risk value.
- CMPLX3: Following these steps requires a lot of mental effort.
- CMPLX4: I can easily understand these steps.

Usefulness

- USE1: This methodology helps me to decide how/why to protect data in a system.
- USE2: This methodology would enhance data protection in system designs.
- USE3: This methodology makes data protection easier.
- USE4: This methodology is not useful.

Social Pressure

- SP1: I do not think people at work would support using this methodology at work.
- SP2: People who are important to me (my colleagues) would support using this methodology at work.

Compatibility with the way developers work

- CWDW1: Following the steps is compatible with the way I develop systems.
- CWDW2: Following the steps would help the way I develop systems.
- CWDW3: Following these steps is not compatible with the way I develop systems

Result Demonstrability

- RESDEM1: The results of following these steps are apparent to me.
- RESDEM2: I have no difficulty telling others about the results of following these steps.
- RESDEM3: I would have difficulty explaining why using the privacy risk value of data may or may not be beneficial.

Behavioural Intention to Use the Model

- IUM1: I would like to follow these steps when I develop system.
- IUM2: I do not think I would follow these steps when I develop systems.

C THE DEFINITION PROVIDED FOR THE ATTRIBUTES OF DATA ELEMENTS

Relatedness: Relatedness of a data element is defined as how relevant a data element is to the purpose of a system. For example, location is highly relevant for an application that shows you the map of your current location. However, use of location is questionable in an application you use to seek medical advice. *Sensitivity*: The level of sensitivity of a particular data element to a user. For example, the name of a person is generally assumed to be less sensitive than the address. Sensitivity is also related to the risk of collecting and storing a particular data element. The higher the sensitivity, the higher the risk. *Visibility*: Visibility of a data element determines the exposure a particular data element gets in an application. Some data elements are expected to be visible in an application while some are not. Replacing the credit card number with dots as a user types it into an on-line form is an example of limiting the visibility of a data element. However, the visibility attribute of a data is strongly related to the sensitivity of data, as the general notion is that highly sensitive data should get low visibility. One should note that this is a determined value and not

an inherent value of the data element before the application is designed. *Granularity*: The level to which a data element could be diffused. Whether the address of a user is required at House number level or at the Country level.

D CONSISTENCY: MODEL CONSTRUCTS

Table 6. Reliability Statistics for Each Construct

Construct	Cronbach's alpha
Complexity	0.714
Usefulness	0.911
Social Pressure	0.749
Compatibility	0.754
Results Demonstrability	0.827
Intention to Use	0.748

E DISCRIMINANT VALIDITY

* significance level is <0.0001 for all pairs.

F CONVERGENT VALIDITY

See Table 7 * significance level is <0.0001 for all pairs.

Table 7. Discriminant Validity among Construct Pairs

Construct Pair	Chi-square	Degrees of Freedom
Intention-Usefulness	64.77	1
Intention-Social Pressure	29.833	1
Intention-Compatibility	54.040	1
Intention-Demonstrability	47.459	1
Intention-Complexity	38.28	1
Demonstrability-Usefulness	54.639	1
Demonstrability-Social Pressure	33.255	1
Demonstrability-Compatibility	36.753	1
Demonstrability-Complexity	38.284	1
Compatibility-Complexity	27.037	1
Compatibility-Usefulness	75.363	1
Compatibility-Social Pressure	43.424	1
Social Pressure-Complexity	10.986	1
Social Pressure-Usefulness	46.080	1
Usefulness-Complexity	20.498	1

G INVITATION EMAIL

Hello XX

We are researchers at XXX. We got your email address by looking at GitHub for popular Javascript-Vue projects and finding contributors with publicly posted email addresses. (We found you by looking at the [url of the repository from which the email was taken]).

Table 8. Convergent Validity

Item	Loading
Usefulness1	0.833
Usefulness2	0.854
Usefulness3	0.771
Usefulness4	0.911
SP1	0.839
SP2	0.634
RESDEM1	0.769
RESDEM2	0.822
RESDEM3	0.711
CWDW1	0.767
CWDW2	0.812
CWDW3	0.816
CMPLX1	0.831
CMPLX2	0.765
CMPLX3	0.705
CMPLX4	0.695
IUM1	0.770
IUM2	0.716

We are looking for programmers like you because we are conducting a study to investigate software developers’ attitudes towards privacy and data protection when they design software systems. We conduct this study in collaboration with the Y.

This is a survey study that would take 20–30 mins and would involve answering questions based on a scenario. It would be great if you could help us out by voluntary participation, for the evolution of research in understanding software developers’ engagement with data privacy.

You can go to the link at [1] and complete the survey. The survey has been tested on Google Chrome, Firefox and Microsoft Edge. The styles may not work properly in other browsers. It can be done on your own computer whenever you have 30 mins to spare and would like to see how you understand data privacy risk as a developer. Once you click the link you will have to complete the study in one go, because we do not store data at intermediary steps.

If you decide to participate, submit the survey and wish to know more about the study or the results please let us know by replying to this email. We will notify you of the results of the study.

Please feel free to forward this email to anyone else (colleagues, friends, etc.) who may be interested/available.

Also, because we care about not spamming people, if you don’t respond to this email or if you tell us you are not interested, we will not email you again or distribute/store your email anywhere.

If you have any questions, please feel free to contact myself at [sender’s email].

[1] Survey Link: [removed for anonymisation of the paper]

Thank you! [sender name and affiliation]

H THE PEM USED IN THE STUDY AND ITS FLOW

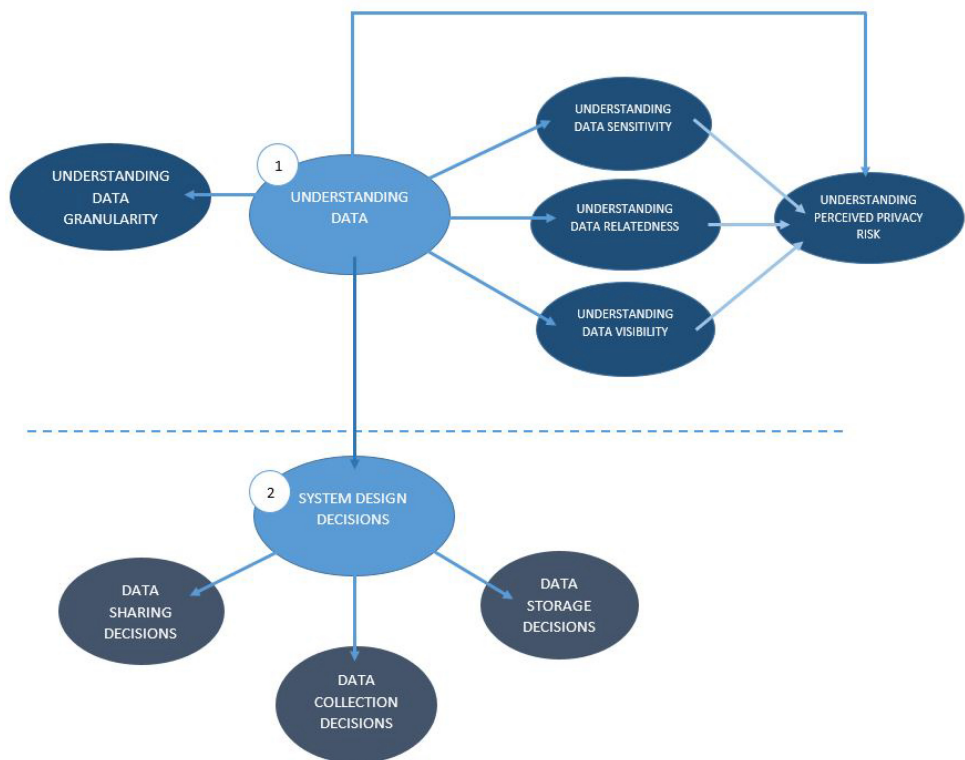


Fig. 5. This is the PEM that is used in the study. This PEM enables making system design decisions to protect data through understanding data.

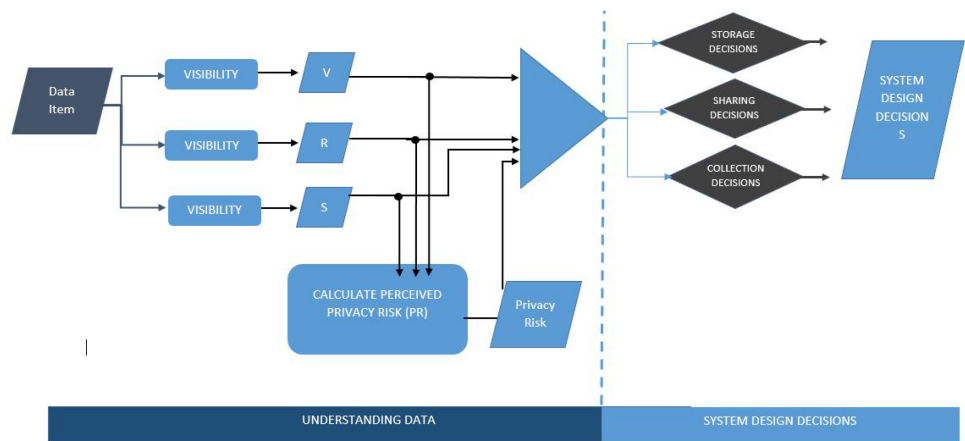


Fig. 6. This is the workflow, or the action steps in the PEM used in the study. These steps were implemented in a web app for the study participants to follow the PEM.

I PARTICIPANT DEMOGRAPHIC QUESTIONS

- Your Age
 - Above 18 up to 24
 - Above 24 up to 32
 - Above 32 up to 40
 - Above 40
- Your Gender
 - Male
 - Female
 - Prefer not to say
- Your Occupation
 - Software Developer
 - Software Architect
 - Owns a software company
 - A Student
 - Freelancer
 - None of the above
- Your Experience in the software industry
 - 0– up to 2 years
 - more than 2 up to 5 years
 - more than 5 up to 10 years
 - more than 10 years

J FREQUENCY OF LIKERT SCALE RESPONSES

Ease of Use is used instead of Complexity according to the questions.

* Please note that the negation questions are converted in this so strongly-disagree in negation is converted to strongly-agree in calculating the frequencies here.

Table 9. Frequencies of Likert Scales

Item	Strongly Disagree	Somewhat Disagree	Neutral	Somewhat Agree	Strongly Agree
Usefulness1	9	23	28	55	34
Usefulness2	8	13	17	71	40
Usefulness3	16	21	32	53	27
Usefulness4	10	16	16	56	51
SP1	13	20	37	56	23
SP2	26	48	45	22	8
RESDEM1	12	24	23	61	29
RESDEM2	10	25	23	51	40
RESDEM3	17	30	16	55	31
CWDW1	11	22	31	56	29
CWDW2	11	17	25	68	28
CWDW3	3	11	26	54	55
CMPLX1	6	13	18	72	40
CMPLX2	15	27	36	42	29
CMPLX3	9	35	37	47	21
CMPLX4	8	11	21	66	43
IUM1	8	12	19	64	46
IUM2	3	8	23	52	63

ACKNOWLEDGMENTS

We acknowledge the voluntary participation of all our participants and thank them for their time and effort.

REFERENCES

- [1] 2015. GitHub blog, 2015 A Closer Look at Europe. Retrieved from: <https://github.blog/2015-06-17-a-closer-look-at-europe/>.
- [2] 2016. European Union Data Protection. Retrieved from: <http://ec.europa.eu/justice/data-protection/data-collection>.
- [3] Yasemin Acar, Michael Backes, Sascha Fahl, Simson Garfinkel, Doowon Kim, Michelle L. Mazurek, and Christian Stransky. 2017. Comparing the usability of cryptographic APIs. In *Proceedings of the IEEE Symposium on Security and Privacy (SP'17)*. IEEE, 154–171.
- [4] Yasemin Acar, Sascha Fahl, and Michelle L. Mazurek. 2016. You are not your developer, either: A research agenda for usable security and privacy research beyond end users. In *Proceedings of the IEEE Cyber Security Development Conference (SecDev'16)*. IEEE.
- [5] Yasemin Acar, Christian Stransky, Dominik Wermke, Michelle L. Mazurek, and Sascha Fahl. 2017. Security developer studies with Github users: Exploring a convenience sample. In *Proceedings of the 13th Symposium on Usable Privacy and Security (SOUPS'17)*. 81–95.
- [6] Ritu Agarwal and Jayesh Prasad. 1997. The role of innovation characteristics and perceived voluntariness in the acceptance of information technologies. *Dec. Sci.* 28, 3 (1997), 557–582.
- [7] Ritu Agarwal and Jayesh Prasad. 2000. A field study of the adoption of software process innovations by information systems professionals. *IEEE Trans. Eng. Manag.* 47, 3 (2000), 295–308.
- [8] Oshrat Ayalon, Eran Toch, Irit Hadar, and Michael Birnhack. 2017. How developers make design decisions about users' privacy: The place of professional communities and organizational climate. In *Proceedings of the ACM Conference on Computer Supported Cooperative Work and Social Computing*. ACM, 135–138.
- [9] Richard P. Bagozzi and Lynn W. Phillips. 1982. Representing and testing organizational theories: A holistic construal. *Admin. Sci. Quart.* 27, 3 (1982), 459–489.
- [10] Rebecca Balebako and Lorrie Cranor. 2014. Improving app privacy: Nudging app developers to protect user privacy. *IEEE Sec. Priv.* 12, 4 (2014), 55–58.
- [11] Henri Barki and Jon Hartwick. 2001. Interpersonal conflict and its management in information system development. *MIS Quart.* 25, 2 (2001), 195–228.
- [12] Kristian Beckers, Stephan Faßbender, Maritta Heisel, and Rene Meis. 2012. A problem-based approach for computer-aided privacy threat identification. In *Proceedings of the Annual Privacy Forum*. Springer, 1–16.
- [13] Victoria Bellotti and Abigail Sellen. 1993. Design for privacy in ubiquitous computing environments. In *Proceedings of the 3rd European Conference on Computer-Supported Cooperative Work (ECSCW'93)*. Springer, 77–92.
- [14] Sean W. Brooks, Michael E. Garcia, Naomi B. Lefkowitz, Suzanne Lightman, and Ellen M. Nadeau. 2017. An introduction to privacy engineering and risk management in federal information systems. *NIST Interagency/Internal Report (NISTIR)-8062* (2017).
- [15] Cormac Callanan, Borka Jerman-Blažič, and Andrej Jerman Blažič. 2016. User awareness and tolerance of privacy abuse on mobile Internet: An exploratory study. *Telemat. Inform.* 33, 1 (2016), 109–128.
- [16] J. C. Cannon. 2004. *Privacy: What Developers and IT Professionals Should Know*. Addison-Wesley Professional.
- [17] Lee J. Cronbach. 1951. Coefficient alpha and the internal structure of tests. *Psychometrika* 16, 3 (1951), 297–334.
- [18] George Danezis, Josep Domingo-Ferrer, Marit Hansen, Jaap-Henk Hoepman, Daniel Le Metayer, Rodica Tîrtea, and Stefan Schiffner. 2015. Privacy and data protection by design—from policy to engineering. *European Union Agency for Network and Information Security*. European Union Agency for Network and Information Security (ENISA).
- [19] Fred D. Davis. 1985. *A Technology Acceptance Model for Empirically Testing New End-user Information Systems: Theory and Results*. Ph.D. Dissertation. Massachusetts Institute of Technology.
- [20] Fred D. Davis, Richard P. Bagozzi, and Paul R. Warshaw. 1989. User acceptance of computer technology: A comparison of two theoretical models. *Manag. Sci.* 35, 8 (1989), 982–1003.
- [21] Edward L. Deci. 1972. Intrinsic motivation, extrinsic reinforcement, and inequity. *J. Person. Soc. Psych.* 22, 1 (1972), 113.
- [22] Mark T. Dishaw and Diane M. Strong. 1999. Extending the technology acceptance model with task–technology fit constructs. *Inform. Manag.* 36, 1 (1999), 9–21.
- [23] M. Eva and S. Guilford. 1996. Committed to a RADical approach? A survey of systems development methods in practice. In *Proceedings of the 4th Conference of the British Computer Society Information Systems Methodologies Specialist Group*. 87–96.

- [24] Georgios Gousios, Andy Zaidman, Margaret-Anne Storey, and Arie Van Deursen. 2015. Work practices and challenges in pull-based development: The integrator's perspective. In *Proceedings of the 37th International Conference on Software Engineering, Volume 1*. IEEE Press, 358–368.
- [25] Daniel Graziotin, Fabian Fagerholm, Xiaofeng Wang, and Pekka Abrahamsson. 2017. Unhappy developers: Bad for themselves, bad for process, and bad for software product. In *Proceedings of the 39th International Conference on Software Engineering*. IEEE Press, 362–364.
- [26] Daniel Graziotin, Fabian Fagerholm, Xiaofeng Wang, and Pekka Abrahamsson. 2018. What happens when software developers are (un) happy. *J. Syst. Softw.* 140 (2018), 32–47.
- [27] Seda Gürses, Carmela Troncoso, and Claudia Diaz. 2011. *Engineering Privacy by Design*. Retrieved from <https://software.imdea.org/~carmela.troncoso/papers/Gurses-CPDP11.pdf>.
- [28] Irit Hadar, Tomer Hasson, Oshrat Ayalon, Eran Toch, Michael Birnhack, Sofia Sherman, and Arod Balissa. 2018. Privacy by designers: Software developers' privacy mindset. *Empir. Softw. Eng.* 23, 1 (2018), 259–289.
- [29] Joseph F. Hair, William C. Black, Barry J. Babin, Rolph E. Anderson, Ronald L. Tatham et al. 2006. In *Multivariate Data Analysis*, Vol. 6.
- [30] Bill C. Hardgrave, Fred D. Davis, and Cynthia K. Riemenschneider. 2003. Investigating determinants of software developers' intentions to follow methodologies. *J. Manag. Inform. Syst.* 20, 1 (2003), 123–151.
- [31] Bill C. Hardgrave and Richard A. Johnson. 2003. Toward an information systems development acceptance model: The case of object-oriented systems development. *IEEE Trans. Eng. Manag.* 50, 3 (2003), 322–336.
- [32] Magda Huisman and Juhani Iivari. 2002. The individual deployment of systems development methodologies. In *Proceedings of the International Conference on Advanced Information Systems Engineering*. Springer, 134–150.
- [33] Shubham Jain and Janne Lindqvist. 2014. Should I protect you? Understanding developers' behavior to privacy-preserving APIs. In *Proceedings of the Workshop on Usable Security (USEC'14)*.
- [34] Carlos Jensen, Joseph Tullio, Colin Potts, and Elizabeth D. Mynatt. 2005. *STRAP: A Structured Analysis Framework for Privacy*. Technical Report. Georgia Institute of Technology.
- [35] Richard A. Johnson, Bill C. Hardgrave, and E. Reed Doke. 1999. An industry analysis of developer beliefs about object-oriented systems development. *ACM SIGMIS Datab.: Datab. Adv. Inform. Syst.* 30, 1 (1999), 47–64.
- [36] Christos Kalloniatis, Evangelia Kavakli, and Stefanos Gritzalis. 2008. Addressing privacy requirements in system design: The PriS method. *Req. Eng.* 13, 3 (2008), 241–255.
- [37] Mohamed Khalifa and June M. Verner. 2000. Drivers for software development method usage. *IEEE Trans. Eng. Manag.* 47, 3 (2000), 360–369.
- [38] Jae-On Kim and Charles W. Mueller. 1978. *Factor Analysis: Statistical Methods and Practical Issues*. Vol. 14. Sage.
- [39] Aniket Kittur, Ed H. Chi, and Bongwon Suh. 2008. Crowdsourcing user studies with Mechanical Turk. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*. ACM, 453–456.
- [40] Rex B. Kline. 2015. *Principles and Practice of Structural Equation Modeling*. Guilford Publications.
- [41] John P. Kotter et al. 1995. Leading change: Why transformation efforts fail. *Harvard Business Review*, May–June 1995 Issue.
- [42] Kenneth A. Kozar. 1989. Adopting systems development methods: An exploratory study. *J. Manag. Inform. Syst.* 5, 4 (1989), 73–86.
- [43] Christopher Kuner. 2007. In *European Data Protection Law: Corporate Regulation and Compliance* (2nd Ed.).
- [44] Saadi Lahlou, Marc Langheinrich, and Carsten Röcker. 2005. Privacy and trust issues with invisible computers. *Commun. ACM* 48, 3 (2005), 59–60.
- [45] Thomas D. LaToza, Gina Venolia, and Robert DeLine. 2006. Maintaining mental models: A study of developer work habits. In *Proceedings of the 28th International Conference on Software Engineering*. ACM, 492–501.
- [46] Paul Legris, John Ingham, and Pierre Colletette. 2003. Why do people use information technology? A critical review of the technology acceptance model. *Inform. Manag.* 40, 3 (2003), 191–204.
- [47] Dorothy Leonard-Barton. 1987. Implementing structured software methodologies: A case of innovation in process technology. *Interfaces* 17, 3 (1987), 6–17.
- [48] Tianshi Li, Yuvraj Agarwal, and Jason I. Hong. 2018. Coconut: An IDE plugin for developing privacy-friendly apps. *Proc. ACM Interact., Mobile, Wear. Ubiqu. Technol.* 2, 4 (2018), 178.
- [49] Kai-Uwe Loser and Martin Degeling. 2014. Security and privacy as hygiene factors of developer behavior in small and agile teams. In *Proceedings of the IFIP International Conference on Human Choice and Computers*. Springer, 255–265.
- [50] Jesus Luna, Neeraj Suri, and Ioannis Krontiris. 2012. Privacy-by-design based on quantitative threat modeling. In *Proceedings of the 7th International Conference on Risks and Security of Internet and Systems (CRiSIS'12)*. IEEE, 1–8.
- [51] Naresh K. Malhotra, Sung S. Kim, and James Agarwal. 2004. Internet users' information privacy concerns (IUIPC): The construct, the scale, and a causal model. *Inform. Syst. Res.* 15, 4 (2004), 336–355.
- [52] Yod Samuel Martín García and José María del Álamo Ramiro. 2017. A metamodel for privacy engineering methods. In *Proceedings of the CEUR Workshop*.

- [53] Kieran Mathieson. 1991. Predicting user intentions: Comparing the technology acceptance model with the theory of planned behavior. *Inform. Syst. Res.* 2, 3 (1991), 173–191.
- [54] Kunal Mohan and Frederik Ahlemann. 2013. Understanding acceptance of information system development and management methodologies by actual users: A review and assessment of existing literature. *Int. J. Inform. Manag.* 33, 5 (2013), 831–839.
- [55] Gary C. Moore and Izak Benbasat. 1991. Development of an instrument to measure the perceptions of adopting an information technology innovation. *Inform. Syst. Res.* 2, 3 (1991), 192–222.
- [56] Emerson Murphy-Hill and Gail C. Murphy. 2011. Peer interaction effectively, yet infrequently, enables programmers to discover new tools. In *Proceedings of the ACM Conference on Computer Supported Cooperative Work*. ACM, 405–414.
- [57] Alena Naiakshina, Anastasia Danilova, Christian Tiefenau, Marco Herzog, Sergej Dechand, and Matthew Smith. 2017. Why do developers get password storage wrong? A qualitative usability study. In *Proceedings of the ACM Conference on Computer and Communications Security (CCS'17)*.
- [58] Jum C. Nunnally, Ira H. Bernstein, and Jos M. F. ten Berge. 1967. *Psychomet. Theor.* Vol. 226. McGraw-Hill New York.
- [59] Marie Caroline Oetzel and Sarah Spiekermann. 2014. A systematic methodology for privacy impact assessments: A design science approach. *Europ. J. Inform. Syst.* 23, 2 (2014), 126–150.
- [60] Michael Perscheid, Benjamin Siegmund, Marcel Taeumel, and Robert Hirschfeld. 2017. Studying the advancement in debugging practice of professional software developers. *Softw. Qual. J.* 25, 1 (2017), 83–110.
- [61] Cynthia K. Riemenschneider, Bill C. Hardgrave, and Fred D. Davis. 2002. Explaining software developer acceptance of methodologies: A comparison of five theoretical models. *IEEE Trans. Softw. Eng.* 12 (2002), 1135–1145.
- [62] Robert Rosenthal and Ralph L. Rosnow. 1975. *Primer of Methods for the Behavioral Sciences*. John Wiley & Sons.
- [63] Awanthika Senarath and Nalin A. G. Arachchilage. 2018. Why developers cannot embed privacy into software systems? An empirical investigation. In *Proceedings of the 22nd Conference of Evaluation and Assessment in Software Engineering (EASE'18)*. ACM, 211–216.
- [64] Awanthika Senarath and Nalin Asanka Gamagedara Arachchilage. 2019. A data minimization model for embedding privacy into software systems. *Comput. Sec.* 87 (2019), 101605.
- [65] Swapneel Sheth, Gail Kaiser, and Walid Maalej. 2014. Us and them: A study of privacy requirements across North America, Asia, and Europe. In *Proceedings of the 36th International Conference on Software Engineering*. ACM, 859–870.
- [66] Dag I. K. Sjøberg, Bente Anda, Erik Arisholm, Tore Dyba, Magne Jørgensen, Amela Karahasanovic, Espen Frimann Koren, and Marek Vokác. 2002. Conducting realistic experiments in software engineering. In *Proceedings of the International Symposium on Empirical Software Engineering*. IEEE, 17–26.
- [67] Sarah Spiekermann and Lorrie Faith Cranor. 2009. Engineering privacy. *IEEE Trans. Softw. Eng.* 35, 1 (2009), 67–82.
- [68] Fareena Sultan and Lillian Chan. 2000. The adoption of new technology: The case of object-oriented computing in software companies. *IEEE Trans. Eng. Manag.* 47, 1 (2000), 106–126.
- [69] Shirley Taylor and Peter A. Todd. 1995. Understanding information technology usage: A test of competing models. *Inform. Syst. Res.* 6, 2 (1995), 144–176.
- [70] Ronald L. Thompson, Christopher A. Higgins, and Jane M. Howell. 1994. Influence of experience on personal computer utilization: Testing a conceptual model. *J. Manag. Inform. Syst.* 11, 1 (1994), 167–187.
- [71] Louis G. Tornatzky and Katherine J. Klein. 1982. Innovation characteristics and innovation adoption-implementation: A meta-analysis of findings. *IEEE Trans. Eng. Manag.* 1 (1982), 28–45.
- [72] Viswanath Venkatesh. 1999. Creation of favorable user perceptions: Exploring the role of intrinsic motivation. *MIS Quart.* 23, 2 (1999), 239–260.
- [73] Viswanath Venkatesh and Fred D. Davis. 2000. A theoretical extension of the technology acceptance model: Four longitudinal field studies. *Manag. Sci.* 46, 2 (2000), 186–204.
- [74] Rolf H. Weber. 2010. Internet of Things—New security and privacy challenges. *Comput. Law Secur. Rev.* 26, 1 (2010), 23–30.
- [75] Dominik Wermke and Michelle Mazurek. 2017. Security developer studies with GitHub users: Exploring a convenience sample. In *Proceedings of the Symposium on Usable Privacy and Security (SOUPS'17)*.
- [76] Glenn Wurster and Paul C. van Oorschot. 2009. The developer is the enemy. In *Proceedings of the Workshop on New Security Paradigms*. ACM, 89–97.
- [77] Kim Wuyts, Riccardo Scandariato, and Wouter Joosen. 2014. Empirical evaluation of a privacy-focused threat modeling methodology. *J. Syst. Softw.* 96 (2014), 122–138.
- [78] Shundan Xiao, Jim Witschey, and Emerson Murphy-Hill. 2014. Social influences on secure development tool adoption: Why security tools spread. In *Proceedings of the 17th ACM Conference on Computer Supported Cooperative Work & Social Computing*. ACM, 1095–1106.
- [79] Yuchen Yang, Longfei Wu, Guisheng Yin, Lijie Li, and Hongbin Zhao. 2017. A survey on security and privacy issues in Internet-of-Things. *IEEE Int. Things J.* 4, 5 (2017), 1250–1258.

- [80] Adriana Zaiț and P. E. Berteau. 2011. Methods for testing discriminant validity. *Manag. Market. J.* 9, 2 (2011), 217–224.
- [81] Gerald Zaltman, Robert Duncan, and Jonny Holbek. 1973. *Innovations and Organizations*. John Wiley & Sons.
- [82] Dongpo Zhang. 2018. Big data security and privacy protection. In *Proceedings of the 8th International Conference on Management and Computer Science (ICMCS'18)*. Atlantis Press.

Received March 2019; revised August 2019; accepted September 2019